

The Company Information Screen — What It Is, Why It Exists, and What It Feeds

Prepared for: Michael Lyman, Owner, Greenway Marijuana **Legal entity:** LYMAN'S MARIJUANA L.L.C. · WA UBI 603 353 555 · EIN 46-4217016 **Slice:** books-31 · **Migration:** 0196_company_profile.sql **Date prepared:** 22 August 2026

The short version

You now have one screen that holds every fact about your business that a tax form, a bank file, or a state report will ever ask for. There are twenty fields. They are read by ten different forms and files. Before this slice, those facts lived in your head, in Sage, on a shelf, and in three unrelated corners of this application, and every form that needed them asked a slightly different question and got a slightly different answer.

The whole point is this: **you type your EIN once**. After that, Form 941 reads it, Form 940 reads it, every W-2 reads it, the W-3 that covers those W-2s reads it, every 1099-NEC reads it, the NACHA file that moves money out of your bank reads it, and the EFTPS deposit reads it. Eight consumers, one keystroke. If it is wrong, it is wrong in one place and you fix it in one place.

I loaded your real Greenway data into the engine as a test. The result was **ready: 10 of 10 forms**. I then ran it with an empty profile and got **ready: 0 of 10**. Both numbers matter. The first says your data is sufficient. The second says the screen is genuinely load-bearing rather than decorative — if it were not really feeding those forms, a blank profile would still have reported some of them ready, and it does not.

Why a screen like this is the first thing real payroll systems build

There is a reason ADP, Paychex, QuickBooks, and Sage all make you complete a company setup wizard before they will let you run a single payroll. Every downstream document is a restatement of the same handful of facts. The IRS says so in plain language. From the *Instructions for Form 941 (2026)*, mirrored in this repository and quoted verbatim in the screen itself:

"Enter your EIN, name, and address in the spaces provided. Also enter your name and EIN on the top of pages 2 and 3."

That instruction appears, in one form or another, on every federal employment return you will ever file. The identity block is not a formality — it is how the IRS decides which taxpayer's account gets credited for the money you send. Which brings us to the single most expensive mistake available in this whole domain, also quoted verbatim from the same instructions:

"Always be sure the EIN on the form you file exactly matches the EIN the IRS assigned to your business. Don't use your social security number (SSN) or individual taxpayer identification number (ITIN) on forms that ask for an EIN."

The reason this is expensive rather than merely embarrassing is that a mismatched EIN does not bounce. The return is accepted. The deposit is accepted. They are simply credited to the wrong account, or to no account, and you find out months later when a notice arrives saying you never filed and never paid. You then get to prove a negative. The screen refuses to let you save an EIN that is not nine digits, and the database refuses it a second time, independently, because I do not trust a browser to be the only thing standing between you and that letter.

The twenty fields and who reads them

I will not list all twenty in a table, because the useful thing is not the list but the pattern. Let me walk the five groups.

Federal identity — EIN, legal name, trade name, entity type, which federal return you file (941 or 944), and your deposit schedule. The legal name is the one that must match your SS-4, not the name on your sign. Your legal name is LYMAN'S MARIJUANA L.L.C. and your trade name is Greenway Marijuana, and the forms want them in different boxes. The *Instructions for Form 941* are explicit that these are two different things, which is exactly why the screen holds them as two different fields rather than one "company name."

Address — street, optional second line, city, state, ZIP, country. Six forms read this block. The W-2 instructions specify what belongs in Box c, and that is the authority the screen cites when it explains the field to you.

Contact and signature — who answers the phone about a filing, and who signs the return. These are not the same person

conceptually even when they are the same person in practice, and the W-3 has separate boxes for them. The signer title field is the one I want to flag for you specifically, because it is the field where your own instinct will be wrong.

State accounts — your ESD account number (000-073905-00-0), your UBI (603 353 555), your L&I account (521,756-00), your L&I risk class (6403, Stores: Specialty), and which state's unemployment tax you pay (WA). The ESD account number requirement is not IRS guidance, it is Washington statute, and the screen quotes RCW 50.12.070 directly.

Calendar and administration — fiscal year end (December), payroll start date (1 January 2027, your cutover), effective dates for legal-name and address changes, the responsible party, and the last four of their SSN.

The one field where your instinct is wrong: signer title

You are the owner. Every instinct says the title is "Owner." For an LLC taxed as an S-corporation, "Owner" is the wrong answer, and the *Instructions for Form 941 (2026)* say why in the Part 5 signature rules. Verbatim:

"Corporation (including an LLC treated as a corporation)—The president, vice president, or other principal officer duly authorized to sign."

"Owner" describes your economic interest in the LLC. It is not a corporate office. Because you elected S treatment, the IRS is looking for a principal officer — president, vice president, or equivalent. That is why the test fixture in this codebase says **President** and not **Owner**, and there is a comment in the test file explaining that I changed it precisely because our own mentor text said "Owner" was wrong and the fixture was contradicting the lesson.

This is the kind of thing the mentor layer exists for. It is not a tooltip telling you that the EIN field wants your EIN. It is the accountant sitting next to you saying "not that one, this one, and here is the sentence from the instructions that says so."

Your entity type is one of nine the system will store, and every one of them now resolves to a signer rule backed by a verbatim quote:

Entity type	Signing authority per IRS Form 941 Part 5
Sole proprietor	The individual who owns the business
Partnership / LLC taxed as partnership	An authorised partner, member, or officer
C-corp, S-corp, LLC taxed as either	President, VP, or other principal officer
Single-member LLC (disregarded)	The owner of the LLC
Trust or estate	The fiduciary

Earlier in this slice, four of those nine returned "I don't know." I had justified that with a standing rule about refusing to classify unknown input, and on review that was the rule being used as an excuse for unfinished work — the IRS text covers all nine cases plainly. I mirrored three more verbatim quotes and widened the logic. It still returns "I don't know" for a genuinely unknown entity type such as a cooperative, because that is a real refusal rather than a cosmetic one, and there is a test that asserts it.

The two cross-checks that protect your money

Two of the checks in this migration are not about a single field being well-formed. They are about two places in your platform disagreeing, which is the failure mode that costs real money and is nearly invisible.

Your EIN versus your ACH originator. The NACHA file that sends direct deposit to your employees carries an "immediate origin" field, conventionally the digit 1 followed by your nine-digit EIN. If your company profile says 46-4217016 and your ACH settings say something else, then your payroll file and your Form 941 are describing two different businesses. I tested this on a real Postgres database: with a deliberately mismatched originator, the audit reported —

EIN disagrees with ach_company_settings.immediate_origin — company_profile.ein is 464217016 and immediate_origin digits are 9999999999. The NACHA header and the Form 941 would name different businesses. Fix whichever is wrong.

Then I repaired it to the correct 1464217016 and the finding disappeared. Both halves matter: a check that always fires is as useless as one that never does.

Your WSLCB licence versus your CCRS export. Same idea. Your licence number 413541 lives in the company profile for identity purposes and in the licence settings for the state traceability export. If they drift apart, your CCRS submissions and

your company record stop matching. Tested the same way, killed the same way.

Notice what the audit does **not** do. It does not pick a winner. It tells you the two values disagree and asks you to fix whichever is wrong, because choosing between two EINs on your behalf would be guessing, and guessing about your federal identity is precisely the thing we do not do.

How I proved this actually works, rather than merely compiles

A green test suite is worth very little on its own. What follows is what I actually ran.

I rebuilt a genuine database from scratch — every migration from 0001 through 0195 in order, 247 tables — and only then applied 0196 for the first time. This matters because a migration that has only ever run on a database that already contained its own tables has not been tested. It applied cleanly, and the audit function returned zero findings. I applied it a second time and a third time: no errors, still zero findings, because you will inevitably re-paste a migration at some point and it must not punish you for that.

Then I tried to break it seven ways, and confirmed each break was caught:

Deleting the company profile row was caught as "row count is not exactly one." Turning off row-level security was caught as "row level security is not enabled on company_profile." Adding a policy that says `using (true)` — a policy that reads like security and provides none — was caught as "policy is not gated on is_owner()." Dropping a policy entirely was caught as "fewer policies than expected." Stripping a column's documentation comment was caught as "column has no comment naming its consumers." And the two disagreement checks above were caught with the exact messages quoted.

Alongside those seven I ran a **control**: the same audit, with nothing broken, which must return zero. It did. Without that control I could not distinguish "the audit catches everything" from "the audit complains about everything."

One of those mutants taught me something. When I tried to insert a *second* company profile row, Postgres refused it outright — a check constraint makes a second row impossible, so that branch of the audit is unreachable from that direction. The reachable risk is *zero* rows, from a bad delete or an incomplete restore, so that is the mutant I ran instead. I left a comment in the test harness recording the wrong turn, because the next person to read it will otherwise wonder why the obvious test is missing.

I also caught myself leaving a mess. My fifth mutant stripped the EIN column's comment, and I forgot to put it back. Two mutants later the audit was still correctly reporting one finding — and because I had a control baseline, I could see the leak instead of mistaking it for noise. Re-applying the migration repaired it. The audit found my own sloppiness, which is the best evidence I can offer that it will find yours.

Finally, the file you actually paste into Supabase is a comment-stripped copy of the real migration, and a stripped copy that has quietly drifted from the original is a trap. So I built a second complete database from the editor-safe copy and compared the two resulting schemas fact by fact: 39 columns, 3 policies, 16 constraints, 1 index, 10 grants, 39 column comments — **108 catalogue rows, all identical**. The file you run and the file I test are the same file.

The full suite is **378 test files and 7,989 tests, all passing**. TypeScript compiles with no errors. The linter reports zero errors, and I verified that all nine of its warnings live in files I never touched.

Your question about migration 0195 and the RLS prompt

You mentioned 0195 ran fine this time but did not ask whether you wanted row-level security enabled, and asked whether that matters.

It does not matter, and nothing needs fixing. Here is the evidence rather than reassurance.

That prompt is a convenience feature of the Supabase dashboard's table editor. When you create a table by clicking through the UI, the dashboard offers to enable RLS for you, because a table left unprotected in an exposed schema is the single most common Supabase security mistake. When you create tables by running SQL — which is what you are doing — the dashboard does not interpose, because your SQL is presumed to say what you meant. It is not a safety net that failed; it is a prompt that only appears on a path you were not using.

The reason it does not matter here is that 0195 enables RLS itself, explicitly, in the SQL. I confirmed this two ways. First, by reading the file: it contains four `enable row level security` statements, one for each of the four tables it creates — `employee_w4`, `employee_i9`, `employee_pay`, and `employee_ssn_reveals`. Second, and more importantly, by querying the live database after the migration ran, because what a file says and what a database did are two different claims:

Table	RLS enabled	Policies attached
employee_i9	yes	4
employee_pay	yes	4
employee_w4	yes	4
employee_ssn_reveals	yes	2

All four protected. The reveal log deliberately has only two policies — select and insert — because it is append-only. There is no update policy and no delete policy, and since RLS denies by default, the *absence* of a policy is what makes that table tamper-resistant. Nobody can edit or erase a record of who looked at a Social Security number, including you.

I checked the same thing for the new company profile table, and while I was there I checked something the Supabase documentation is emphatic about and which is easy to get wrong. Their guidance states plainly: "*Adding policies doesn't take those grants back. A table protected only by policies still hands anon an insert path if you never revoke the grant.*" In other words, enabling RLS and writing policies is not sufficient on its own — the underlying table grants have to be right too. So I looked:

Role	Privileges on company_profile
anon	none
authenticated	SELECT, INSERT, UPDATE
postgres	full (server-side only)

No grant at all for anonymous visitors. No DELETE grant for anyone through the API, matching the deliberate absence of a delete policy — your company identity record cannot be deleted through the application, only corrected. Three policies, every one gated on `is_owner()`, verified by the audit and by the mutation test that adds a `using (true)` policy and watches the audit catch it.

So: the missing prompt is a non-event, and I would rather have RLS in the migration file than in a dialog box. A dialog box protects the one table you happened to create while looking at it. A migration protects the table on every database it is ever applied to, including the one you restore from backup at 2am having forgotten the dialog existed.

The one genuine observation worth recording is that the audit function is owner-only, which means it refused to run for me at first — I had to create a real owner record and impersonate it to get the zero-findings result. That is the gate working. It also means that when you want to check your own books, you can, and nobody else can.

What this unlocks, and what is still missing

This screen is the foundation for the pipeline you described: time cards to payable hours to gross to net to ACH to the general ledger to the forms. The identity half of that chain is now done and tested. Being straight with you about the rest:

Your time punches are recorded and your ACH settings are recorded, and the EIN that ties them together is now verified against the books. What does not exist yet is the piece in the middle that turns punches into a pay period's payable hours, including overtime. Your withholding engine can already compute the federal taxes on a paycheck, but the year-to-date accumulators are not persisted yet, and a pay run currently stores one lump tax figure rather than the split that a W-2 and a 941 require. There are no W-2, W-3, 941, 940, or 5208 builders yet. The pay run screen still asks you to type net pay, which it should be calculating.

None of that is blocked by anything, and all of it reads from the screen you just got. That was the point of building this one first.

Sources

All quotations above are verbatim from documents mirrored into this repository and checked by an automated verifier on every commit. That verifier currently confirms 188 quotations against local copies of their sources; it fails the build if a single character drifts.

- Internal Revenue Service, *Instructions for Form 941, Employer's QUARTERLY Federal Tax Return* (2026). <https://www.irs.gov/pub/irs-pdf/i941.pdf>
- Internal Revenue Service, *General Instructions for Forms W-2 and W-3* (2026). <https://www.irs.gov/pub/irs-pdf/iw2w3.pdf>

- Internal Revenue Service, *Instructions for Form 940, Employer's Annual Federal Unemployment (FUTA) Tax Return* (2025). <https://www.irs.gov/pub/irs-pdf/i940.pdf>
- Internal Revenue Service, *Publication 15, (Circular E), Employer's Tax Guide* (2026). <https://www.irs.gov/pub/irs-pdf/p15.pdf>
- Revised Code of Washington, RCW 50.12.070, *Employer records — Reports*. <https://app.leg.wa.gov/rcw/default.aspx?cite=50.12.070>
- Supabase, *Row Level Security* documentation. <https://supabase.com/docs/guides/database/postgres/row-level-security>

One honest gap in that citation list: the Washington L&I quarterly report instructions are not yet mirrored, so the L&I account number and risk class fields carry no authority citation. That is recorded as a debt in the code, and there is a test asserting the debt exists so it cannot be quietly forgotten.